V. Sarcar, *Java Design Patterns*
https://doi.org/10.1007/978-1-4842-7971-7_20

20. Memento Pattern

Vaskaran Sarcar[1]
(1) Garia, Kolkata, India

This chapter covers the Memento pattern.

GoF Definition

Without violating encapsulation, the Memento pattern captures and externalizes an object's internal state so that the object can be restored to this state later.

Concept

As per the dictionary, the word *memento* is used as a reminder of past events. Following the object-oriented way, you can also track (or save) the states of an object. So, whenever you want to restore an object to its previous state, you can consider using this pattern.

In this pattern, you commonly see three participants called memento, originator, and caretaker (often used as a client). The workflow can be summarized as follows: the originator object has an internal state, and a client can set a state in it. To save the current internal state of the originator, a client (or caretaker) requests a memento from it. A client can also pass a memento (which it holds) back to the originator to restore a previous state. Following the proper approach, these saving and restoring operations do not violate encapsulation.

Real-Life Example

You can see a classic example of the Memento pattern in the states of a finite state machine. It is a mathematical model, but one of its simplest applications is a turnstile. A turnstile has rotating arms, which initially are locked. When you go through it (for example, after putting some coins in), the locks open and the arms rotate. Once you pass through, the arms return to a locked state.

You can also consider a game application that has many difficulty levels. If it is a long-running game, it's useful if you can pause at a certain level and later resume the game. If you want, you can directly jump to a preferred level, too.

> **Note** Using the real-world examples, sometimes it is tough to differentiate between the patterns. For example, the turnstile example can be used in the context of the State pattern (Chapter 22), too. But once you go through the State pattern, you'll understand that the state in the State pattern is not used like a memento. In a State pattern, you deal with an

object's behavior in a particular state. But you can use a memento to transform an object into a desired state to enable that particular behavior. Let's analyze a case study: when a television is in its switched-on mode, you can switch between channels; otherwise, you cannot change the channels. But you can switch off the television by pressing the power button on a remote control. So, in this case, a power button press helps a television to change between its states, which in turn shows different behaviors of a television in those states.

Computer World Example

In a drawing application, you may need to revert to an older state. You can also consider the transactions in a database where you may need to roll back a specific transaction. Memento patterns can be used in those scenarios. Here is another example:

- Consider a Java Swing example. You may see the use of the `JTextField` class, which extends the abstract class `JTextComponent` and provides undo support mechanism. The Java documentation at **https://docs.oracle.com/en/java/javase/16/docs/api/java.desktop/javax/swing/text/JTextComponent.html** says the following about this capability: "Support for an edit history mechanism is provided to allow undo/redo operations. The text component does not itself provide the history buffer by default but does provide the `UndoableEdit` records that can be used in conjunction with a history buffer to provide the undo/redo support. The support is provided by the Document model, which allows one to attach `UndoableEditListener` implementations." In this case an implementation of `javax.swing.undo.UndoableEdit` acts like a memento and an implementation of `javax.swing.text.Document` acts like an originator and `javax.swing.undo.UndoManager` acts as a caretaker.

`java.io.Serializable` is often referred to as an example of a memento but it should be noted that though you can serialize a memento object, it is not a mandatory requirement for the Memento design pattern.

Implementation

Before you proceed further, here are some important suggestions from the GoF:

- You see three important players (memento, originator, and caretaker) in this design implementation.
- A memento is an object that stores the internal state of another object. We call this "another object" as the memento's originator. It means the originator has an internal state which is our concern.
- The originator initializes the memento with its current state. Ideally, the originator that produces the memento can access its internal state.
- Only the originator can store and retrieve necessary information from the memento. This memento is "opaque" to other objects.
- A caretaker class is the container of mementos. This class is used for the memento's safekeeping, but it never operates or examines the content of a memento. A caretaker can get the memento from the originator.
- A caretaker first asks the originator for a memento object. Then it can set a new state to it. Next, if it wants, it can save the memento. To reset an originator's state, it passes back a memento object to the originator.

- The originator uses the memento to restore its internal state.
- The caretaker sees a *narrow* interface to the memento. It can only pass the memento to other objects. The originator, in contrast, sees a *wide* interface, one that lets it access all the data necessary to restore itself to its previous state. Ideally, only the originator that produced the memento is permitted to access the memento's internal state.

> **Note** In short, the caretaker is not allowed to make any changes to a memento. It simply passes the memento to an originator. From its standpoint, it sees a narrow interface to the memento. On the contrary, the originator sees a wide interface because it has full access to the memento. The originator uses this facility to restore its previous state.

A Memento design pattern can have varying implementations using different techniques. In this chapter, you'll see two demonstrations. Demonstration 1 is relatively simple and easy to understand. It is improved upon in Demonstration 2. In both implementations, I **don't** use a separate caretaker class. Instead, I use the client code to play the role of the caretaker.

In the upcoming example, the `Originator` class has a field called `state`. This variable represents the current state of an `Originator` object, which is your main concern. In addition, the `Originator` class has three methods called `setState(...)`, `restoreMemento(...)`, and `getMemento()`.

The first one is used to set a new state of an originator. It is defined as follows (remember that you are setting a state, but not saving this state yet):

```
public void setState(String newState) {
 this.state = newState;
 System.out.println(" Originator's current state is: " + state);
}
```

The second one is used to restore the originator to a previous state. This state is contained in a memento (that comes as a method argument) from the caretaker. A caretaker sends this memento object that it saved earlier. This method is defined as follows:

```
public void restore(Memento memento) {
this.state = memento.state;
System.out.println("Restored to state: " + state);
}
```

The third one is used to supply a memento in response to a caretaker's request. It contains the current state of the `Originator` object. It is defined as follows:

```
public Memento getMemento() {
  Memento currentMemento = new Memento(state);
  return currentMemento;
}
```

To keep the memento class short and simple, there's only one variable. To match the originator state, it's called `state` as well. In many other applications, you may see a memento class with the getter-setter methods (or, at least a getter method). You don't use them here because you're omitting the use of an access modifier for this field. This variable has package-private visibility, which means that other classes in the same package can access it, so it is slightly more open than private visibility but not as much as protected visibility. You can guess the reason behind this design: you do not want an

arbitrary object to set a new value or access the internal state of the memento. In addition, you make this field `final` so that this value cannot be modified later. (You'll see an additional discussion on this in Q&A 20.1). You make this class `public` so that the caretaker/client can see it from outside of the package. Here is the `Memento` class:

```
public class Memento {
    // package-private visibility with final keyword
    final String state;//prevents editing the state later
    // package-private visibility
      Memento(String state) {
            this.state = state;
      }
 }
```

Keep in mind that in this example, the client class acts as the caretaker that is responsible for the memento's safe keeping, but it never operates on the content of a memento. Here the caretaker holds an `Originator` object and asks for memento objects from it. So, you see the following lines inside the client code:

```
Originator originator = new Originator();
memento = originatorObject.getMemento();
```

This caretaker also saves and holds the mementos in a list. So, you see the following lines of code inside the client code:

```
List<Memento> savedMementos = new ArrayList<Memento>();
```

This time you create a separate method to mark the checkpoints (or snapshots) and save them into the `savedMementos` list. This is optional. (I use it to avoid code duplication inside the client code and promote the Don't Repeat Yourself (DRY) principle.) Here is the method definition:

```
private static void saveSnapShot(Originator originator, String checkPoint, List<Memento
     // Setting a new state
     originator.setState(checkPoint);
     // Get the current state from the
     // originator through a memento
     Memento memento = originator.getMemento();
     System.out.println(".Saving this checkpoint.");
     savedMementos.add(memento);
}
```

Now you understand why you see the following line when you create the first snapshot (marked with `Snapshot #0`):

```
saveSnapShot(originator, "Snapshot #0", savedMementos);
```

In the same way, you create and save other snapshots except the last one. For the last snapshot (`Snapshot #4`), you see the following line where the client simply sets a new state to the originator but does not save this state:

```
originator.setState("Snapshot #4");
```

Finally, notice that inside `Client.java`,there are additional import statements because you use the `List` and `ArrayList` data structures to store mementos. You know that the `ArrayList` class implements the `List` interface. In this example, you could use `ArrayList` only, but you know programming to an interface is a better practice. This is the reason for the line

```
List<Memento> savedMementos = new ArrayList<Memento>();
```

instead of the line

```
ArrayList<Memento> savedMementos = new ArrayList<Memento>();
```

The remaining code is easy to understand. So, let's proceed.

Class Diagram

Figure 20-1 shows the class diagram. The originator creates mementos, so you see the <<create>> tag with the usage edge in the diagram.

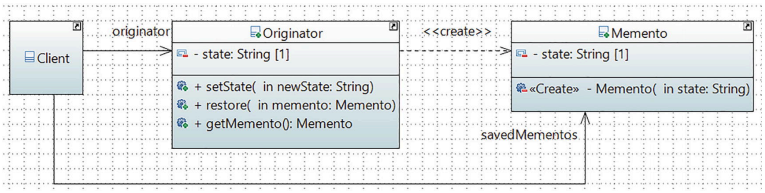

Figure 20-1 Class diagram

Package Explorer View

Figure 20-2 shows the high-level structure of the program. You can see that `Memento.java` and `Originator.java` are placed inside `implementation_1`, but `Client.java` is placed outside of the folder.

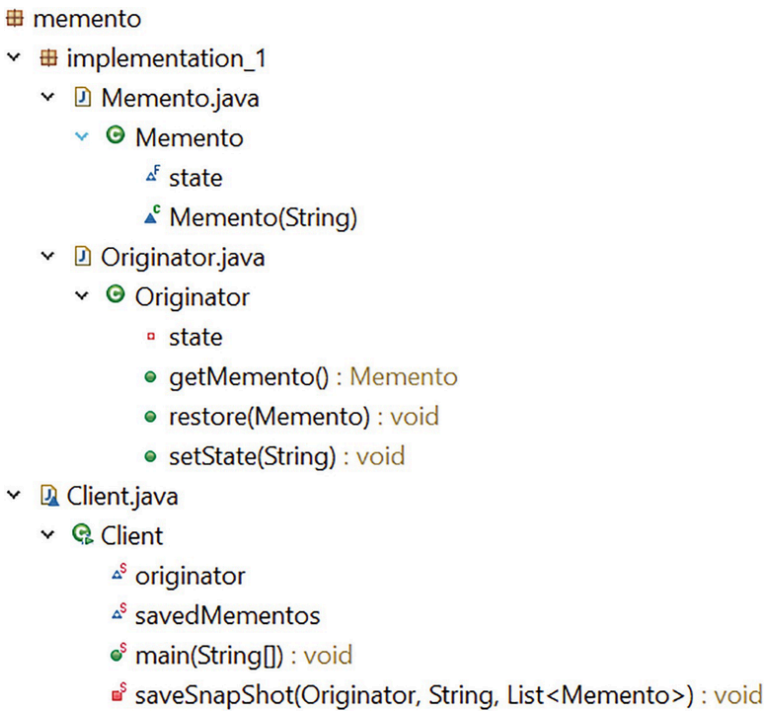

Figure 20-2 Package Explorer view

Demonstration 1

Here is the complete implementation:

```
// Memento.java
package jdp3e.memento.implementation_1;
public class Memento {
      final String state;
          Memento(String state) {
```

```
            this.state = state;
      }
 }
// Originator.java
package jdp3e.memento.implementation_1;
/**
 * This is the originator class. As per GoF:
 * 1. It creates a memento that contains a snapshot of
 *    its current internal state.
 * 2. It uses a memento to restore its internal state.
 */
class Originator {
      private String state;
      // Setting a new internal state
      public void setState(String newState) {
            this.state = newState;
            System.out.print("The current state is " + state);
      }
      // Back to an old state (Restore)
      public void restore(Memento memento) {
            // The following line is ok due
            // to package-private visibility
            this.state = memento.state;
            System.out.println("Restored to state: " + state);
      }
       // Originator (which contains its current state)
       // will supply the memento in response to the
       // caretaker's request.
      public Memento getMemento() {
            Memento currentMemento = new Memento(state);
            return currentMemento;
      }
}
// Client.java
package jdp3e.memento;
import java.util.ArrayList;
import java.util.List;
import jdp3e.memento.implementation_1.Memento;
import jdp3e.memento.implementation_1.Originator;
/**
 * This is the caretaker class. As per GoF:
 * 1. This class is responsible for memento's safe-keeping.
 * 2. Never operates or Examines the content of a Memento.
 */
class Client {
      static Originator originator;
      static List<Memento> savedMementos;
      public static void main(String[] args) {
            System.out.println("***Memento Pattern Demonstration-1.***\n");
            originator = new Originator();
            // This list stores the checkpoints
            savedMementos = new ArrayList<Memento>();
            // Snapshot #0
            saveSnapShot(originator, "Snapshot #0", savedMementos);
            // Snapshot #1
            saveSnapShot(originator, "Snapshot #1", savedMementos);
            // Snapshot #2
            saveSnapShot(originator, "Snapshot #2", savedMementos);
            // Snapshot #3
            saveSnapShot(originator, "Snapshot #3", savedMementos);
            // Snapshot #4. Taking a snapshot,
            // but not adding as a restore point.
            originator.setState("Snapshot #4");
            // Undo's
```

```
            // Roll back everything...
            System.out.println("\n\nStarted restoring process...");
            for (int i = savedMementos.size(); i > 0; i--) {
                  // Get a restore point
                  originator.restore(savedMementos.get(i - 1));
            }



            // Redo's
            System.out.println("\nPerforming redo's now.");
            // Restore starts from "Snapshot #1" now.
            for (int i = 2; i <= savedMementos.size(); i++) {
                  // Get a restore point
                  originator.restore(savedMementos.get(i - 1));
            }
            // Restoring to any specified checkpoint
            System.out.println("\nRestoring to Snapshot #1.");
            originator.restore(savedMementos.get(1));
      }
      private static void saveSnapShot(Originator
              originator,String checkPoint, List<Memento>
                                          savedMementos) {
            // Setting a new state
            originator.setState(checkPoint);
            // Get the current state from the
            // originator through a memento
            Memento memento = originator.getMemento();
            System.out.println(". Saving this checkpoint.");
            savedMementos.add(memento);
      }
}
```

Note In Java, the `java.util` and `java.awt` packages define a type called List. So, do not forget to use the correct one, which is shown here. In addition, when you download the source code from the Apress website, you'll see additional code comments that I already discussed. I kept them from your immediate reference.

Output

Here's the output:

```
***Memento Pattern Demonstration-1.***
The current state is Snapshot #0. Saving this checkpoint.
The current state is Snapshot #1. Saving this checkpoint.
The current state is Snapshot #2. Saving this checkpoint.
The current state is Snapshot #3. Saving this checkpoint.
The current state is Snapshot #4
Started restoring process...
Restored to state: Snapshot #3
Restored to state: Snapshot #2
Restored to state: Snapshot #1
Restored to state: Snapshot #0
Performing redo's now.
Restored to state: Snapshot #1
Restored to state: Snapshot #2
Restored to state: Snapshot #3
Restoring to Snapshot #1.
Restored to state: Snapshot #1
```

Analysis

Per the concept of this program, you can use three different variations of undo operations:

- You can just go back to the previous restore point.
- You can go back to your specified restore point directly using the `index` property. For example, to go back directly to Snapshot #1, you use the following code snippet:

```
// Restoring to any specified checkpoint
System.out.println("\nRestoring to Snapshot #1.");
originator.restore(savedMementos.get(1));
```

- You can revert all restore points (which is shown using a `for` loop and the `index` property).

> **Note** If an application is using a Memento pattern and there is a state that is a mutable reference type, you may see the implementation of the deep copy technique to store the state inside the Memento object. You learned about deep copies when I discussed the Prototype pattern in Chapter 5.

Q&A Session

20.1 What will happen if I put the client class with the originator and memento class inside the same package? I also see you make the `state` field `final` inside the `Memento` class. Can you explain the reason behind this design?

In this case, the following line inside the client code will not cause any compilation error:

```
      Memento m1=new Memento("test");
```

This means that the client (who plays the role of the caretaker in this example) can create the mementos with an arbitrary state.

Not only that, if I do not mark the state field in the `Memento` class with the `final` keyword, the following line can also work too (if the state field in the `Memento` class is `public`):

```
      m1.state="arbitrary state";
```

So, the client cannot edit/change this state if it is marked `final` in the `Memento` class.

20.2 What are the key advantages of using the Memento design pattern?

Here are some advantages:

- The biggest advantage is that you can always discard the unwanted changes and restore them to an intended or stable state.
- You do not compromise the encapsulation associated with the key objects that are participating in this model.

- You can maintain high cohesion.
- It provides an easy recovery technique.

20.3 What are the key challenges associated with the Memento design pattern?

Here are some disadvantages:

- Having more mementos requires more storage. They also put an additional burden on a caretaker.
- The previous point increases maintenance costs.
- You cannot ignore the time it takes to save these states, which can decrease the overall performance of the application.

It is useful to note that in a language such as C# or Java, you may prefer to use serialization/deserialization techniques instead of directly implementing the Memento design pattern. Each technique has its own pros and cons, but you can combine both techniques in your application.

20.4 I am confused. To support undo operations, which pattern should I prefer, Memento or Command?

The GoF says that they are related patterns. They also said: Commands can use mementos to maintain state for undoable operations. So, it primarily depends on how you want to handle the situation. Suppose you are adding 25 to an integer. After this addition operation, you can undo it by doing a reverse operation. In simple words, 50 + 25 = 75, so to go back, you do 75 – 25 = 50. In this type of operation, you do not need to store the previous state.

But consider a situation where you may need to store the state of your objects before the operation. In this case, you use Memento. For example, in a paint application, you can avoid the cost of undoing some painting operations by storing the list of objects before executing the commands. This stored list can be treated as mementos, and you can keep this list with the associated commands. A similar concept applies to a long-running game application that has multiple levels and in which you may save your last performance level. So, a particular application can use both patterns to support undo operations.

In the end, you must remember that storing a memento object is mandatory in the Memento pattern so that you can revert back to a previous state, but in the case of the Command pattern, it is not necessary to store the commands. Once you execute a command, its job is done. Particularly so, if you do not support undo operations, you may not be interested to store these commands at all.

20.5 I see that mementos are intended not to be read/updated by a client or a caretaker. Why do we need this constraint for a memento class?

If you allow a memento to be read or updated by others (except the originators), you expose the originator's state to the outside world, which means the encapsulation is broken.

20.6 I understand that by using the package-private visibility you avoid the use of getter/setter methods. But in many applications, I no-

tice the use of a getter method inside a Memento class. Does it not violate the encapsulation?

There is a lot of discussion and debate about whether you should use getter/setter or not, particularly when you consider encapsulation. I believe that it depends on how much strictness you want to impose. For example, if you provide getters/setters for all fields without any reason, that is surely a bad design. The same thing applies when you use all the public fields inside the objects. But sometimes the accessor methods are really required and useful. In this book, I aim to encourage you to learn design patterns via simple examples. If I need to consider each minute detail like this, you may lose interest in this subject.

20.7 In many applications, I notice that the memento class is presented as an inner class of Originator. Why are you not following that approach?

A Memento design pattern can be implemented in many different ways (for example, using package-private visibility or using object serialization techniques). But in each case, if you analyze the key aim, you will find that once the memento instance is created by an originator, except its creator, no one else (including caretaker/client) is allowed to access the internal state. A caretaker's job is to store the memento instance (restore points, in our example) and supply them when you are in need.

In demonstration 1, I show a simple way to implement a Memento design pattern. If you wish to see an alternative implementation and prefer to implement the memento class as an inner class of the originator, consider using demonstration 2.

Additional Implementation

Here is an alternative implementation of the Memento design pattern. These are the important changes in this program:

- This time `Memento` is an inner class. It is nested inside the `Originator` class.
- The `Memento` class constructor is `private`. I made this change because I put the caretaker class (`Client2.java`) in the same package. As a result, the caretaker class cannot instantiate the `Memento` class. In short, you cannot use the following lines inside the client code. I kept some supporting comments before each line of code for your easy understanding.

```
// Client can't create the mementos
// and access the state
// Error, because the Memento class is not visible now
// Memento m1=new Memento("test");
// Error, because Memento constructor is private
Originator.Memento m1 = originator. new Memento("test");
// The state variable is final, you cannot edit it
m1.state = "arbitrary state";
```

- The caretaker (`Client2.java`) is very similar to the one in demonstration 1 except this time you need to use `Originator.Memento` instead of `Memento`.

- In demonstration 1, you declare the variables `originator` and `savedMementos` outside of the `main()` method. This time you declare and use them inside the `main()` method. (You can follow either approach.)

Let's go through demonstration 2 now.

Class Diagram

Figure 20-3 shows the modified class diagram.

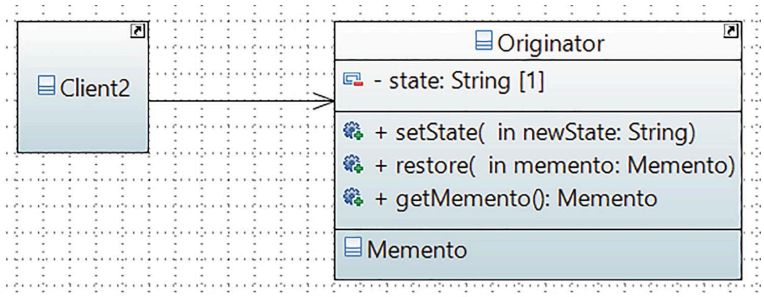

Figure 20-3 Class diagram for demonstration 2

Package Explorer View

Figure 20-4 shows the modified high-level structure of the program.

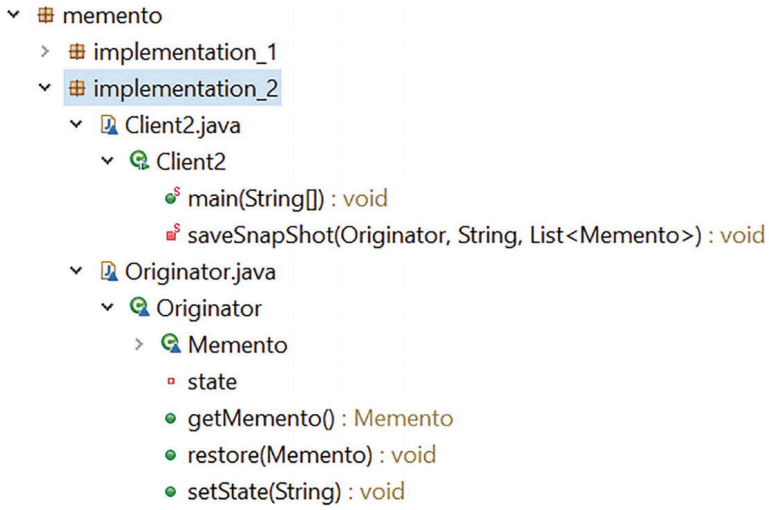

Figure 20-4 Package Explorer view for demonstration 2

Demonstration 2

Here's the modified implementation:

```
// Originator.java
package jdp3e.memento.implementation_2;
class Originator {
      private String state;
      // Setting a new internal state
      public void setState(String newState) {
            this.state = newState;
            System.out.print("The current state is " + state);
      }
      // Back to an old state (Restore)
      public void restore(Memento memento) {
            this.state = memento.state;
            System.out.println("Restored to state: " + state);
```

```
        }
        public Memento getMemento() {
              Memento currentMemento = new Memento(state);
              return currentMemento;
        }
         class Memento {
              final String state;
              private Memento(String state) {
                    this.state = state;
              }
        }
}
// Client2.java
package jdp3e.memento.implementation_2;
import java.util.ArrayList;
import java.util.List;
class Client2 {
        public static void main(String[] args) {
              System.out.println("***Memento Pattern Demonstration-2.***\n");
              Originator originator = new Originator();
              // This list stores the checkpoints
              List<Originator.Memento> savedMementos = new
                            ArrayList<Originator.Memento>();
              // Snapshot #0
              saveSnapShot(originator, "Snapshot #0", savedMementos);
              // Snapshot #1
              saveSnapShot(originator, "Snapshot #1", savedMementos);
              // Snapshot #2
              saveSnapShot(originator, "Snapshot #2", savedMementos);
              // Snapshot #3
              saveSnapShot(originator, "Snapshot #3", savedMementos);
              // Snapshot #4. Taking a snapshot,
              // but not adding as a restore point.
              originator.setState("Snapshot #4");
              // Undo's
              // Roll back everything...
              System.out.println("\n\nStarted restoring process...");
              for (int i = savedMementos.size(); i > 0; i--) {
                    // Get a restore point
                    originator.restore(savedMementos.get(i - 1));
                  // Update the list if required.
              }
              // Redo's
              System.out.println("\nPerforming redo's now.");
              // Restore starts from "Snapshot #1" now.
              for (int i = 2; i <= savedMementos.size(); i++) {
                    // Get a restore point
                    originator.restore(savedMementos.get(i - 1));
                    // Update the list if required.
              }
              // Restoring to any specified checkpoint
              System.out.println("\nRestoring to Snapshot #1.");
              originator.restore(savedMementos.get(1));
        }
        private static void saveSnapShot(Originator originator,
                   String checkPoint, List<Originator.Memento>
                                              savedMementos) {
              // Setting a new state
              originator.setState(checkPoint);
              // Get the current state from the
              // originator through a memento
              Originator.Memento memento =
                                    originator.getMemento();
              System.out.println(". Saving this checkpoint.");
```

```
            savedMementos.add(memento);
        }
}
```

Output

Here is the output. You can see that apart from the initial console message, the output of demonstration 1 and demonstration 2 is the same, but programmatically there are more constraints in this example.

```
***Memento Pattern Demonstration-2.***
The current state is Snapshot #0. Saving this checkpoint.
The current state is Snapshot #1. Saving this checkpoint.
The current state is Snapshot #2. Saving this checkpoint.
The current state is Snapshot #3. Saving this checkpoint.
The current state is: Snapshot #4
Started restoring process...
Restored to state: Snapshot #3
Restored to state: Snapshot #2
Restored to state: Snapshot #1
Restored to state: Snapshot #0
Performing redo's now.
Restored to state: Snapshot #1
Restored to state: Snapshot #2
Restored to state: Snapshot #3
Restoring to Snapshot #1.
Restored to state: Snapshot #1
```

Analysis

I hope that you have a clear idea about the Memento design pattern from these examples. I just want you to note a point: you may notice that once you revert to an old state, you do not removed the state from the saved list. This is because you use this list when you perform redo operations. So, depending on the need, you may need to implement complex undo and redo operations.

Previous chapter
< 19. Command Pattern

Next chapter
21. Strategy Pattern >